A Multi-Component NLP System for Fake News and Misinformation Detection

Week 2 Checkpoint Report

Rian Cortez, Ryence Cortez, Ranz Emmanuel Cuarto, Mark Darius David

Holy Angel University — School of Computing

6INTELSY Intelligent Systems, 2nd Semester AY 2025-2026

1. Data Acquisition and Preparation

The LIAR dataset was successfully downloaded and loaded across all three splits: 10,240 training samples, 1,284 validation samples, and 1,267 test samples. The data pipeline handles TSV loading, label normalization, and text cleaning for both the Text-CNN and DistilBERT branches.

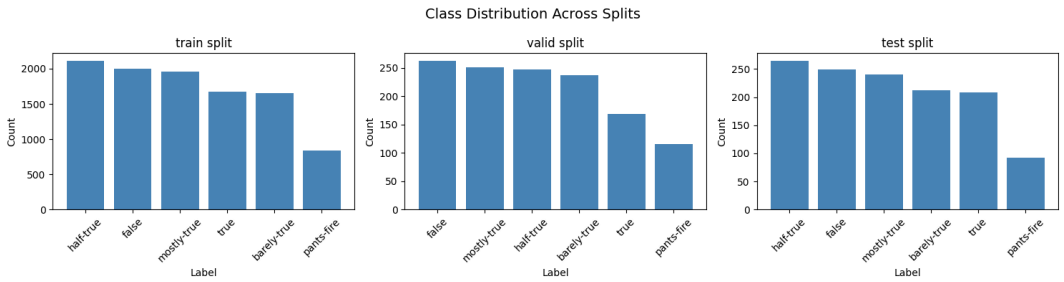
Class distribution is reasonably balanced across all splits, with half-true being the most common label and pants-fire the least frequent at around 8% across splits. Statement lengths average 18 words, with a maximum of 467 words and a minimum of 2 words — the short average length is consistent with the political claim format of the dataset. Republicans are the most represented party (4,497 statements) followed by Democrats (3,336), with Barack Obama being the most frequently mentioned speaker across training data.

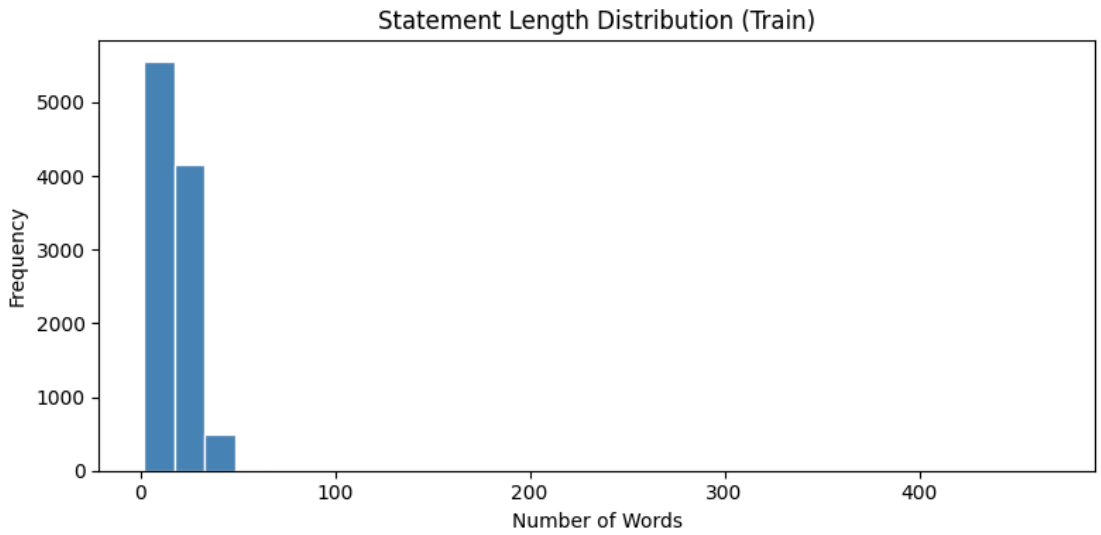
Code:

```
# src/data_pipeline.py

import os

import pandas as pd


COLUMNS = [

    "id", "label", "statement", "subject", "speaker",

    "speaker_job", "state", "party", "barely_true_count",

    "false_count", "half_true_count", "mostly_true_count",

    "pants_fire_count", "context"

]


LABEL_MAP = {
```

```
    "pants-fire": 0, "false": 1, "barely-true": 2,
    "half-true": 3, "mostly-true": 4, "true": 5
}

DATA_DIR = os.path.join(os.path.dirname(__file__), "..", "data", "raw")

def load_split(split="train"):
    filepath = os.path.join(DATA_DIR, f"{split}.tsv")
    df = pd.read_csv(filepath, sep="\t", header=None, names=COLUMNS)
    df = df.dropna(subset=["statement", "label"])
    df = df[df["label"].isin(LABEL_MAP.keys())]
    df["label_id"] = df["label"].map(LABEL_MAP)
    df["statement"] = df["statement"].str.strip()
    return df.reset_index(drop=True)

def load_all_splits():
    splits = {}
    for split in ["train", "valid", "test"]:
        splits[split] = load_split(split)
        print(f"[{split}] loaded: {len(splits[split])} samples")
    return splits
```

Screenshot:





2. Baseline Results

Two baselines were trained and evaluated on the held-out test set.

The non-DL baseline is a Logistic Regression classifier using TF-IDF bigram features with a vocabulary of 10,000 terms. It achieves a test accuracy of 0.2478 and a test Macro-F1 of 0.2214. Per-class performance is weakest on pants-fire (F1: 0.08), which is consistent with that class being the least represented in training.

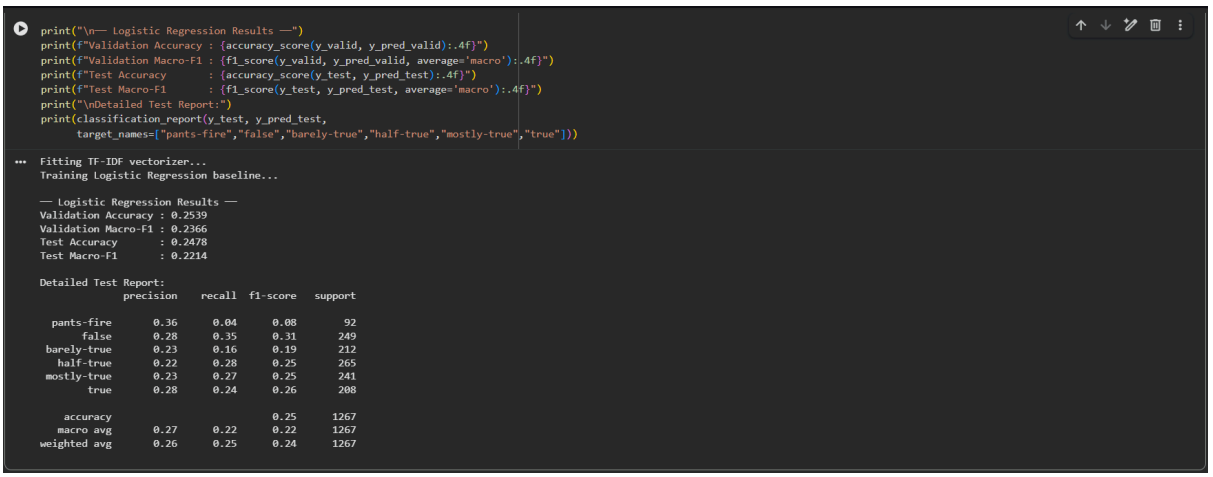
Code:

```
# Logistic Regression baseline (TF-IDF)

from sklearn.linear_model import LogisticRegression

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics import f1_score, accuracy_score


tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))

X_train_tfidf = tfidf.fit_transform(X_train)

X_test_tfidf  = tfidf.transform(X_test)


lr = LogisticRegression(max_iter=1000, random_state=42)

lr.fit(X_train_tfidf, y_train)

y_pred_test = lr.predict(X_test_tfidf)
```

print(f"Test Accuracy : {accuracy_score(y_test, y_pred_test):.4f}")

print(f"Test Macro-F1 : {f1_score(y_test, y_pred_test, average='macro'):.4f}")

Screenshot:



```
print("\n── Logistic Regression Results ──")
print(f"Validation Accuracy : {accuracy_score(y_valid, y_pred_valid):.4f}")
print(f"Validation Macro-F1 : {f1_score(y_valid, y_pred_valid, average='macro'):.4f}")
print(f"Test Accuracy       : {accuracy_score(y_test, y_pred_test):.4f}")
print(f"Test Macro-F1       : {f1_score(y_test, y_pred_test, average='macro'):.4f}")
print("\nDetailed Test Report:")
print(classification_report(y_test, y_pred_test,
      target_names=["pants-fire","false","barely-true","half-true","mostly-true","true"]))
```

```
Fitting TF-IDF vectorizer...
Training Logistic Regression baseline...

── Logistic Regression Results ──
Validation Accuracy : 0.2539
Validation Macro-F1 : 0.2366
Test Accuracy       : 0.2478
Test Macro-F1       : 0.2214

Detailed Test Report:
              precision    recall  f1-score   support

   pants-fire       0.36      0.04      0.08        92
        false       0.28      0.35      0.31       249
  barely-true       0.23      0.16      0.19       212
    half-true       0.22      0.28      0.25       265
  mostly-true       0.23      0.27      0.25       241
         true       0.28      0.24      0.26       208

     accuracy                           0.25      1267
    macro avg       0.27      0.22      0.22      1267
 weighted avg       0.26      0.25      0.24      1267
```

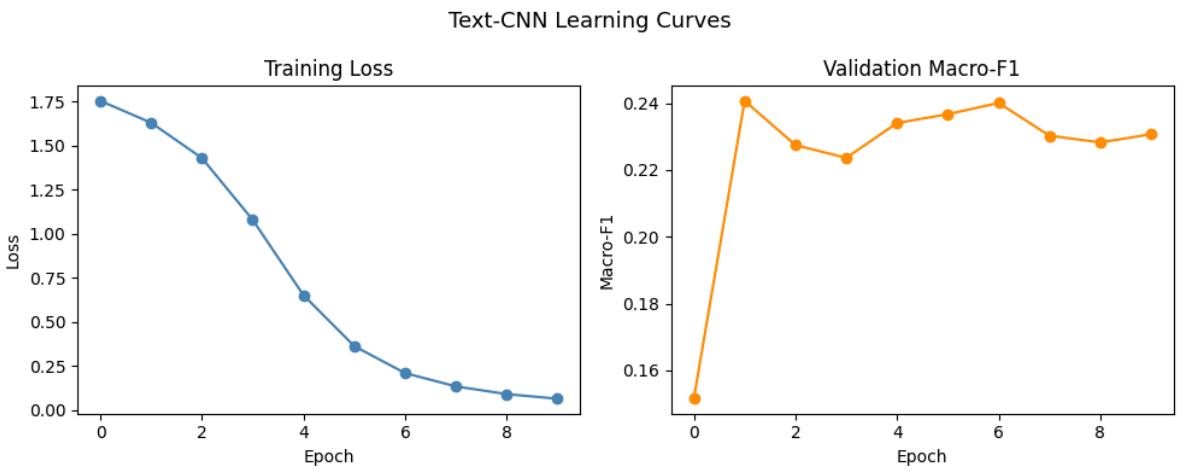
The DL baseline is a Text-CNN following Kim (2014), implemented from scratch with three parallel convolutional filter banks over 300-dimensional GloVe embeddings. It achieves a test accuracy of 0.2478 and a test Macro-F1 of 0.2384, outperforming the Logistic Regression baseline on Macro-F1. Training loss decreases steadily across 10 epochs, however validation F1 peaks at epoch 2 (0.2407) and fluctuates afterward, indicating overfitting. Early stopping will be applied in Week 3 to address this.

Code:

```
# src/models/text_cnn.py

import torch

import torch.nn as nn



class TextCNN(nn.Module):

    def __init__(self, vocab_size, embed_dim, num_classes,

                 kernel_sizes=[3,4,5], num_filters=128, dropout=0.5):

        super().__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        self.convs = nn.ModuleList([
```

```
        nn.Conv1d(embed_dim, num_filters, k) for k in kernel_sizes
    ])
    self.dropout = nn.Dropout(dropout)
    self.fc = nn.Linear(num_filters * len(kernel_sizes), num_classes)

def forward(self, x):
    x = self.embedding(x).permute(0, 2, 1)
    x = [torch.relu(conv(x)).max(dim=2)[0] for conv in self.convs]
    x = torch.cat(x, dim=1)
    x = self.dropout(x)
    return self.fc(x)
```

Screenshot:



Model	Test Accuracy	Test Macro-F1
Logistic Regression (TF-IDF)	0.2478	0.2214

Text-CNN (GloVe 300d)	0.2478	0.2384
DistilBERT (fine-tuned)	TBD	TBD

3. NLP Component — Named Entity Recognition

The NER module was scaffolded using spaCy's en_core_web_sm model and run over the full test set. Of the 1,267 test statements, 38.9% contain at least one PERSON entity, 23.8% contain an ORG entity, and 30.9% contain a GPE (location) entity. Barack Obama and Donald Trump are the most frequently mentioned persons. Medicare and Congress are the most common organizations. Texas is the most mentioned location.

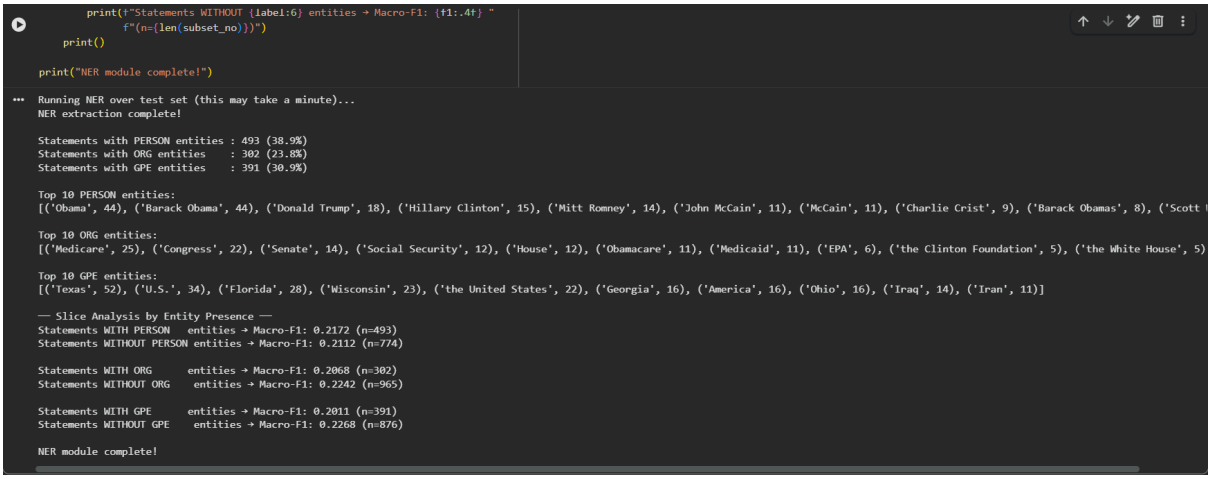
A slice analysis was conducted using Logistic Regression predictions to examine whether classification performance varies by entity presence. Statements referencing specific locations (GPE Macro-F1: 0.2011) and organizations (ORG Macro-F1: 0.2068) perform worse than statements without those entities, suggesting the model struggles more with claims tied to specific institutional or geographic contexts.

Code:

```
# src/models/ner_module.py

import spacy

from collections import Counter


nlp = spacy.load("en_core_web_sm")


def extract_entities(text):

    doc = nlp(text)

    entities = {"PERSON": [], "ORG": [], "GPE": [], "OTHER": []}

    for ent in doc.ents:

        if ent.label_ in entities:

            entities[ent.label_].append(ent.text)

        else:

            entities["OTHER"].append(ent.text)
```

```
    return entities


def run_ner_pipeline(df):
    df = df.copy()
    df['entities']   = df['statement'].apply(extract_entities)
    df['has_person'] = df['entities'].apply(lambda x: len(x['PERSON']) > 0)
    df['has_org']    = df['entities'].apply(lambda x: len(x['ORG']) > 0)
    df['has_gpe']    = df['entities'].apply(lambda x: len(x['GPE']) > 0)
    return df
```

Screenshot:



```
        print(f"Statements WITHOUT {label:6} entities → Macro-F1: {f1:.4f} "
              f"(n={len(subset_no)})")
    print()

print("NER module complete!")

Running NER over test set (this may take a minute)...
NER extraction complete!

Statements with PERSON entities : 493 (38.9%)
Statements with ORG entities    : 302 (23.8%)
Statements with GPE entities    : 391 (30.9%)

Top 10 PERSON entities:
[('Obama', 44), ('Barack Obama', 44), ('Donald Trump', 18), ('Hillary Clinton', 15), ('Mitt Romney', 14), ('John McCain', 11), ('McCain', 11), ('Charlie Crist', 9), ('Barack Obamas', 8), ('Scott

Top 10 ORG entities:
[('Medicare', 25), ('Congress', 22), ('Senate', 14), ('Social Security', 12), ('House', 12), ('Obamacare', 11), ('Medicaid', 11), ('EPA', 6), ('the Clinton Foundation', 5), ('the White House', 5)

Top 10 GPE entities:
[('Texas', 52), ('U.S.', 34), ('Florida', 28), ('Wisconsin', 23), ('the United States', 22), ('Georgia', 16), ('America', 16), ('Ohio', 16), ('Iraq', 14), ('Iran', 11)]

── Slice Analysis by Entity Presence ──
Statements WITH PERSON   entities → Macro-F1: 0.2172 (n=493)
Statements WITHOUT PERSON entities → Macro-F1: 0.2112 (n=774)

Statements WITH ORG      entities → Macro-F1: 0.2068 (n=302)
Statements WITHOUT ORG    entities → Macro-F1: 0.2242 (n=965)

Statements WITH GPE      entities → Macro-F1: 0.2011 (n=391)
Statements WITHOUT GPE    entities → Macro-F1: 0.2268 (n=876)

NER module complete!
```

4. RL Component — Contextual Bandit Agent

A contextual bandit agent with epsilon-greedy exploration was implemented and trained over 500 episodes. The agent selects a confidence threshold from a discretized action space of nine values between 0.1 and 0.9. The reward function penalizes false negatives at -2.0 and false positives at -0.5, reflecting the asymmetry of the detection task.

After training, the agent converged on a threshold of 0.8, selecting it 378 out of 500 times. The bandit agent accumulates higher cumulative reward than a fixed threshold baseline of 0.5 across all episodes. Note that the agent currently runs on simulated softmax outputs sampled from a Dirichlet distribution. It will be re-evaluated with real DistilBERT probability outputs in Week 3.

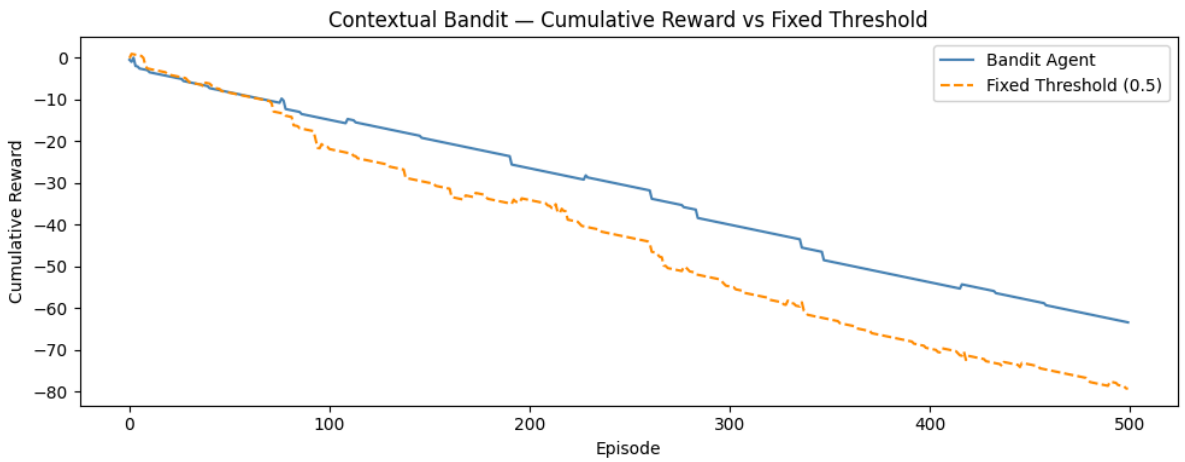
Code:

```
# src/rl_agent.py
```

```
import numpy as np


THRESHOLDS = np.round(np.arange(0.1, 1.0, 0.1), 2)


def get_reward(pred_label, true_label, threshold, max_prob):
    if max_prob < threshold:
        return -0.1
    if pred_label == true_label:
        return 1.0
    if true_label in [0, 1] and pred_label not in [0, 1]:
        return -2.0
    return -0.5


class ContextualBandit:
    def __init__(self, n_actions, epsilon=0.1):
        self.n_actions     = n_actions
        self.epsilon       = epsilon
        self.q_values      = np.zeros(n_actions)
        self.action_counts = np.zeros(n_actions)


    def select_action(self):
        if np.random.random() < self.epsilon:
            return np.random.randint(self.n_actions)
        return np.argmax(self.q_values)


    def update(self, action, reward):
```

```
self.action_counts[action] += 1
self.q_values[action] += (
    (reward - self.q_values[action]) /
    self.action_counts[action]
)
```

Screenshot:



Updated Ethics Statement:

Risk 1: False positives suppressing legitimate speech. Any classifier operating on political content will produce false positives, and in this domain a false positive means labeling a true or contested statement as misinformation. If this system were used to filter content at scale, the downstream harm to free expression could be significant. The mitigation is twofold: first, the system will always be documented as an assistive tool for human reviewers, never as an autonomous decision-maker; second, the default operating threshold will be set conservatively. This concern is reinforced by our Week 2 findings where current models achieve a test Macro-F1 of only 0.22 to 0.24, meaning a significant portion of predictions are wrong at this stage. The contextual bandit agent has already learned to set a conservative threshold of 0.8, effectively abstaining on low-confidence predictions rather than guessing.

Risk 2: Political bias encoded in the model. LIAR is drawn from PolitiFact, which has its own editorial tendencies, and any model trained on it may learn spurious correlations between political affiliation and predicted veracity. This will be examined directly through per-slice F1 analysis broken down by speaker party. Whatever disparities are found will be documented in the Model Card without softening. In Week 2, EDA confirmed a meaningful speaker imbalance in the training set where Republican-affiliated statements (4,497) outnumber Democrat-affiliated ones (3,336). Additionally, NER slice analysis revealed that statements mentioning specific locations (GPE Macro-F1: 0.2011) and organizations (ORG Macro-F1: 0.2068) are consistently harder to classify correctly, suggesting the model may perform unevenly across different contextual framings of political content. Full party-level slice analysis on DistilBERT predictions is planned for Week 3.

Risk 3: Misuse as a censorship tool. A working misinformation classifier is a dual-use artifact. The same system built for research purposes could in principle be repurposed for automated suppression of political content. This risk is addressed by including explicit intended-use documentation restricting the system to research and fact-checker assistance contexts, and by adding a model license that prohibits commercial or governmental censorship applications. This risk is further amplified by the low current model performance where a system with Macro-F1 around 0.22 making autonomous moderation decisions would cause substantial harm, reinforcing why human oversight is non-negotiable for any deployment of this system